

Multi-GPU Parallelism

DiRAC GPU Performance Portability Workshop

Miren Radia

University of Cambridge

Jamie Quinn

DiRAC Training Team

Wednesday 16 September 2026

Introduction

What we will cover

- [Very brief] recap of the Message Passing Interface (MPI)
- How to distribute work to GPUs using MPI
- GPU-aware MPI
- Optimization techniques to improve performance of multi-GPU applications:
 - Binding MPI processes to specific CPU cores, GPUs and NICs
 - Packing kernels
 - Overlapping computation and communication

DiRAC

Training material

<https://tinyurl.com/dirac-multi-gpu>

The material for this course is available in [this repository](#) on GitHub. You will need to clone it to your CSD3 training account in order to complete the exercises:

```
git clone https://github.com/mirenradia/dirac-gppt-multi-gpu.git
```

These slides can be found at tinyurl.com/dirac-multi-gpu. Press the **M** key to view the menu and navigate to specific slides.

MPI Recap

What is MPI?

- The Message Passing Interface (MPI) is a *standard* for distributed memory parallel programming.
- It defines an application programming interface (API) to exchange messages between processes that don't [necessarily] share memory for C and Fortran:
 - Point to point communication e.g. `MPI_Send`, `MPI_Recv`
 - Collective communication e.g. `MPI_Bcast`, `MPI_Reduce`, `MPI_Alltoall`
- Third-party support for other languages e.g. Python (mpi4py) and Julia (MPI.jl)
- Version 5.0 of the standard was released on 5 June 2025 and defined ABI compatibility between MPI implementations allowing users to switch between implementations without recompiling their code¹.
- Various different implementations e.g.:
 - Open MPI based: Open MPI, Nvidia HPC-X
 - MPICH based: MPICH, Intel MPI, Cray MPI, MVAPICH

¹ Currently only MPICH v5.0+ has support for this but older MPICH based implementations have had ABI compatibility between each other for a while

Example MPI program (C++)

```
1 #include <mpi.h>
2 #include <iostream>
3
4 int main(int argc, char* argv[])
5 {
6     MPI_Init(&argc, &argv);
7
8     int world_size;
9     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
10
11     int world_rank;
12     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
13
14     std::cout << "Hello from rank " << world_rank << " of "
15               << world_size << std::endl;
16
17     MPI_Finalize();
18 }
```

Compile with

```
mpicxx -o hello_mpi hello_mpi.cpp
```

Example MPI program (Fortran)

```
1 program hello_mpi
2   use mpi_f08
3   implicit none
4
5   integer :: world_size, world_rank
6
7   call MPI_Init()
8   call MPI_Comm_size(MPI_COMM_WORLD, world_size)
9   call MPI_Comm_rank(MPI_COMM_WORLD, world_rank)
10
11  print *, "Hello from rank ", world_rank, " of ", world_size
12
13  call MPI_Finalize()
14 end program hello_mpi
```

Note we are using the modern `mpi_f08` module which should be preferred over the older `mpi` module and `mpif.h` header file.

Compile with

```
mpifort -o hello_mpi hello_mpi.f90
```

Launching MPI programs

- How you launch MPI programs depends on the system and you should always check the system documentation for details.
- Often you can use **mpiexec** or **mpirun** to launch an MPI program¹:

```
mpiexec -n <num_processes> ./my_mpi_program <options>
```

- Running our previous example with 4 processes would look like:

```
mpiexec -n 4 ./hello_mpi
```

and we would expect output similar to

```
Hello from rank 0 of 4  
Hello from rank 1 of 4  
Hello from rank 2 of 4  
Hello from rank 3 of 4
```

1. When MPI is launched in a batch job, assuming it has been configured correctly, it can usually deduce the number of processes and there is no need to specify **-n <num_processes>**

MPI references

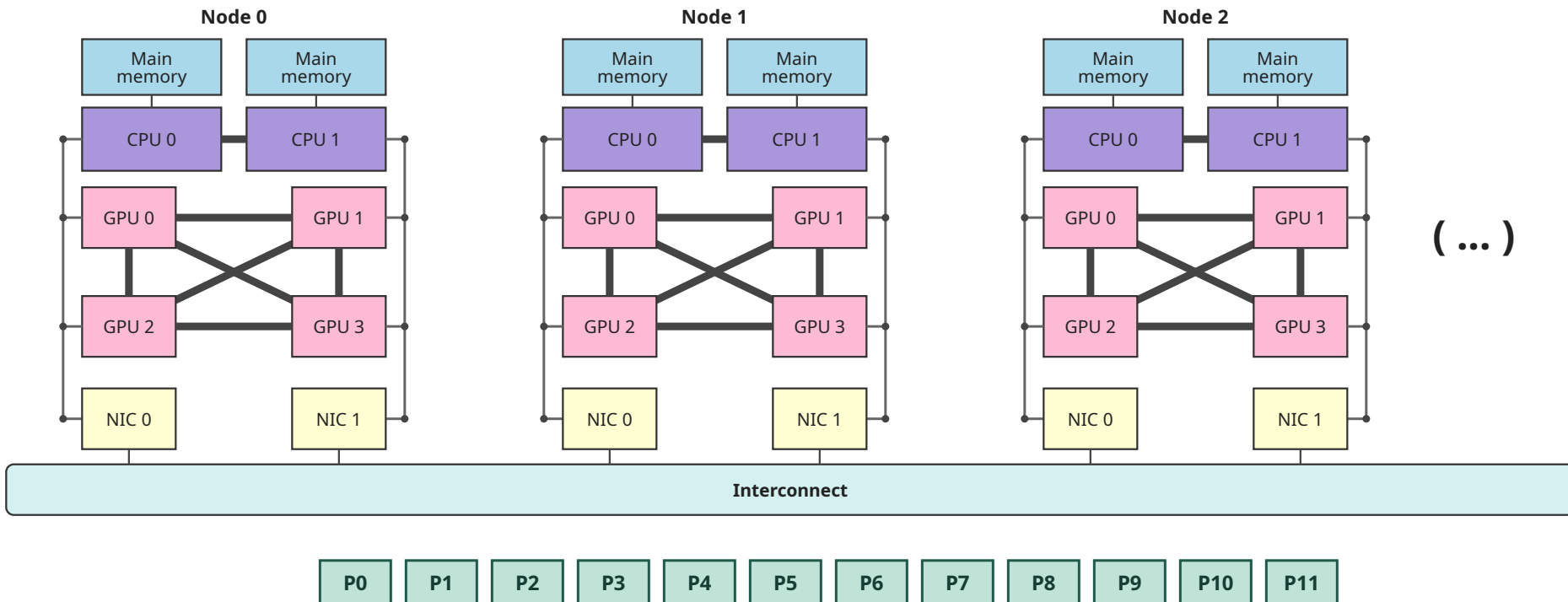
- [MPI Forum Documentation](#)
- [Open MPI Documentation](#)
- [MPICH Man Pages](#)
- [Rookie HPC MPI Docs](#)¹

Any questions?

Distributing work to GPUs

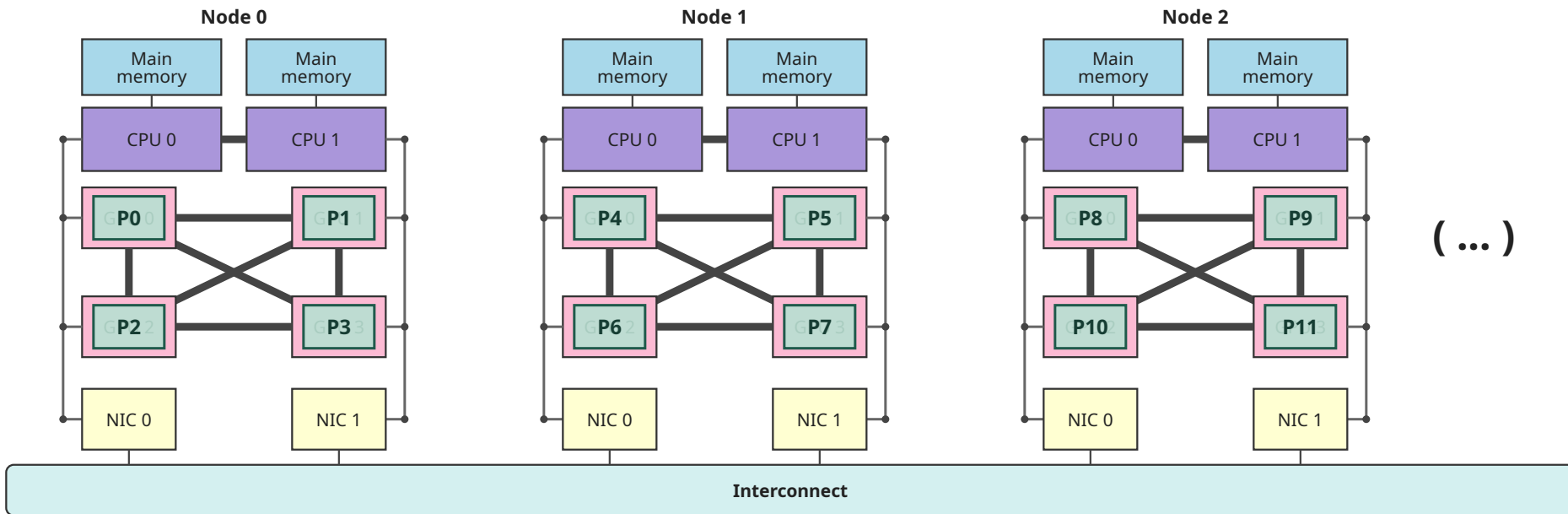
Mapping MPI processes to GPUs

Best practice: one MPI process per GPU.



Mapping MPI processes to GPUs

Best practice: one MPI process per GPU.



Distributing GPUs to local MPI ranks (CUDA/HIP)

```
1 int world_rank;
2 MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
3 MPI_Comm local_comm;
4 MPI_Comm_split_type(MPI_COMM_WORLD, MPI_COMM_TYPE_SHARED, world_rank,
5                     MPI_INFO_NULL, &local_comm);
6
7 int local_rank = -1;
8 MPI_Comm_rank(local_comm, &local_rank);
9 MPI_Comm_free(&local_comm);
10
11 int num_devs = 0;
12 cudaGetDeviceCount(&num_devs);
13 cudaSetDevice(local_rank % num_devs);
```

Distributing GPUs to local MPI ranks (OpenMP)

```
1 use mpi_f08
2 use omp_lib
3
4 integer :: world_rank, local_rank, num_devs
5 type(MPI_Comm) :: local_comm
6
7 call MPI_Comm_rank(MPI_COMM_WORLD, world_rank)
8 call MPI_Comm_split_type(MPI_COMM_WORLD, MPI_COMM_TYPE_SHARED, world_rank, &
9                           MPI_INFO_NULL, local_comm)
10
11 call MPI_Comm_rank(local_comm, local_rank)
12 call MPI_Comm_free(local_comm)
13
14 num_devs = omp_get_num_devices()
15 call omp_set_default_device(mod(local_rank, num_devs))
```

Why one MPI process per GPU?

- Why not more GPUs per MPI process?
 - Each GPU has its own memory space and it makes sense to use MPI to communicate data between GPUs both intra-node and inter-node.
 - You would need to handle memory transfers between GPUs in the same MPI process manually (e.g. using CUDA/HIP or OpenMP) so would have two different methods of communicating data between GPUs.
- Why not more MPI processes per GPU?
 - This is possible if the work of a single MPI process is too small to fully utilize a whole GPU.
 - On Nvidia GPUs, to use multiple MPI processes per GPU efficiently, you need to use the Multi-Process Service (MPS) to reduce context switching overheads (check docs/HPC user support if system has MPS available).
 - On AMD GPUs, this is not necessary.
 - Best to start with one MPI process per GPU and only try more if necessary.
- Some GPUs (e.g. AMD MI250X and Intel Data Center GPU Max 1550/PVC) are essentially two GPUs fused together so it is often best here to use two MPI processes per GPU.

Any questions?

GPU-aware MPI

What is GPU-aware MPI?

- GPU-aware MPI implementations allow you to pass GPU pointers directly to MPI routines without having to copy data to host memory first.
- Often referred to as “CUDA-aware MPI” for Nvidia GPUs.
- Not part of the standard so implementation-dependent.
 - If available it is usually optional so you need to check if implementation has been built with it.
 - Should be available on all GPU HPC systems.
 - Best practice: keep copying data to host memory as a fallback.
- Open MPI is the most common implementation that can be built with GPU support for Nvidia or AMD GPUs (usually provided by the lower level UCX library). We will focus on it from now on.
- Consult system documentation for details: you may need to set environment variables to enable support:
 - `MPICH_GPU_SUPPORT_ENABLED=1` for MPICH and Cray MPI.
 - `I_MPI_OFFLOAD=1` for Intel MPI.
 - `MV2_USE_CUDA=1` or `MV2_USE_ROCM=1` for MVAPICH2.

No GPU-aware MPI (CUDA/HIP)

Without GPU-aware MPI, data transfers between host and GPU memory must be handled manually (or rely on managed/unified memory):

```
1  size_t size = 1024;
2  double* d_ptr;
3  double* h_ptr;
4  cudaMalloc(&d_ptr, size * sizeof(double));
5  cudaMallocHost(&h_ptr, size * sizeof(double));
6
7  <some CUDA kernel that fills d_ptr with data>
8
9  int dest_rank = 1;
10 int tag = 0
11
12 cudaMemcpy(h_ptr, d_ptr, size * sizeof(double), cudaMemcpyDeviceToHost);
13 MPI_Send(h_ptr, size, MPI_DOUBLE, dest_rank, tag, MPI_COMM_WORLD);
```

DiRAC

<https://tinyurl.com/dirac-multi-gpu>

With GPU-aware MPI (CUDA/HIP)

With CUDA/HIP, it's as simple as just passing a pointer to GPU memory to an MPI routine:

```
1  size_t size = 1024;
2  double* d_ptr;
3  cudaMalloc(&d_ptr, size * sizeof(double));
4
5  <some CUDA kernel that fills d_ptr with data>
6
7  int dest_rank = 1;
8  int tag = 0;
9
10 MPI_Send(d_ptr, size, MPI_DOUBLE, dest_rank, tag, MPI_COMM_WORLD);
```

Passing GPU pointers to MPI (OpenMP)

With OpenMP, it's necessary to add the following directive to ensure the GPU pointer is passed:

```
1 integer, parameter :: size = 1024
2 double precision, allocatable :: d_array(:)
3 integer :: dest_rank = 1, tag = 0
4
5 allocate(d_array(size))
6 !$omp target enter data map(alloc: d_array)
7
8 <some OpenMP kernel that fills d_array with data>
9
10 !$omp target data use_device_addr(d_array)
11 call MPI_Send(d_array, size, MPI_DOUBLE, dest_rank, tag, MPI_COMM_WORLD)
12 !$omp end target data
```

How to build GPU-aware MPI programs

All examples below are for Open MPI but can use other MPI implementations by suitably replacing **OMPI_*** environment variables.

- CUDA C++ (with **g++** as host compiler):

```
OMPI_CXX=nvcc mpicxx -x cu --ccbin=g++ --forward-unknown-to-host-compiler <nvcc and g++ flags> \  
-o my_mpi_cuda_program my_mpi_cuda_program.cpp
```

- HIP C++: Set **OMPI_CXX=hipcc** or **OMPI_CXX=amdclang++** and then use **mpicxx** as the compiler e.g.:

```
OMPI_CXX=amdclang++ mpicxx -x hip --offload-arch=<gpu arch> -o my_mpi_hip_program my_mpi_hip_program.cpp
```

- Nvidia Fortran compiler with OpenMP:

```
OMPI_FC=nvfortran mpifort -mp=gpu -gpu=<gpu arch> -o my_mpi_openmp_program my_mpi_openmp_program.f90
```

- AMD Flang compiler with OpenMP:

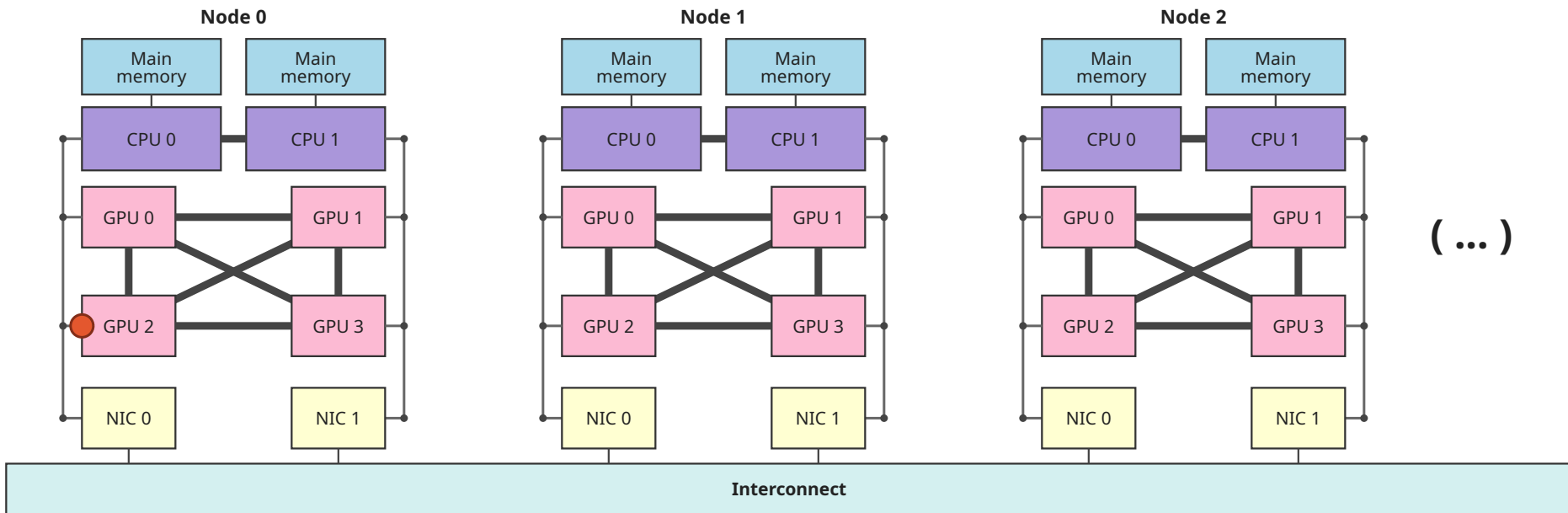
```
OMPI_FC=amdflang mpifort -fopenmp -fopenmp-targets=<gpu arch> -o my_mpi_openmp_program my_mpi_openmp_program.f90
```

What's happening under the hood?

- Simple and naive GPU-aware implementation (or fallback):
 - MPI implementation detects GPU pointer and copies data to host memory before sending it over the network.
 - On the receiving side, data is copied from host memory to GPU memory.
- Most GPU-aware MPI implementations use a more efficient approach that allows data to be sent directly between GPU memory without going through the host:
 - For GPUs on the same node, this can be done using peer-to-peer (P2P) memory access.
 - For GPUs on different nodes, this can be done using remote direct memory access (RDMA) over the network.
- Nvidia: GPUDirect P2P and GPUDirect RDMA (GDR)
- AMD: RDMA and P2P are part of the AMD kernel driver.

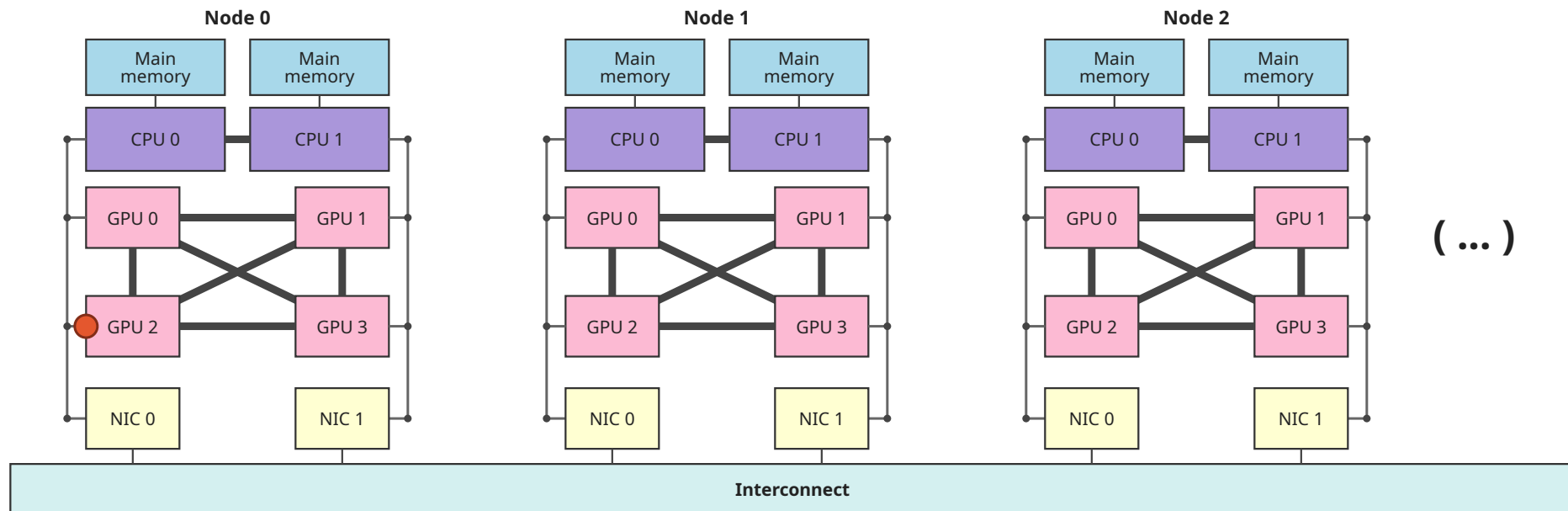
No GPU-aware MPI

Without GPU-aware MPI, data must be copied to host memory. There is also some additional overhead in copying data to/from pinned memory.



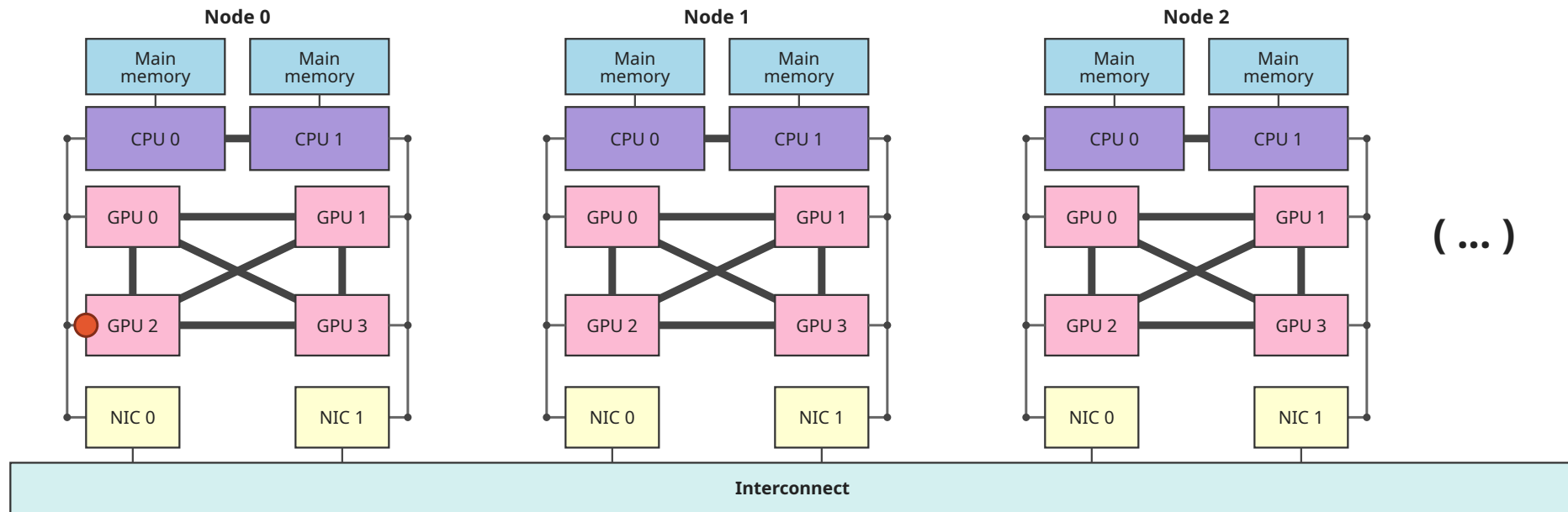
GPU-aware MPI without P2P/RDMA

Even without P2P/RDMA, GPU-aware MPI is still more efficient than non-GPU-aware MPI as the MPI library can use lower-level optimizations to copy data between GPU and host memory.



GPU-aware MPI with P2P/RDMA

GPU-aware MPI with P2P/RDMA is the most efficient as it allows data to be sent directly between GPU memory without going through host memory.



Example: Jacobi solver

- This example solves the 2D Laplace equation in a rectangle:

$$\nabla^2 u = 0 \text{ on } \Omega = (0, L_x) \times (0, L_y)$$

- We use Dirichlet boundary conditions on the left/right boundaries and periodic boundary conditions on the top/bottom boundaries:

$$u(0, y) = u_0, \quad u(L_x, y) = u_1, \quad u(x, 0) = u(x, L_y)$$

- We discretize the domain using a uniform grid and use the Jacobi method to iteratively solve for u at each grid point. At iteration k , the value of u at grid point (x_i, y_j) is computed as follows:

$$u_{i,j}^{(k+1)} = \frac{1}{4} \left(u_{i-1,j}^{(k)} + u_{i+1,j}^{(k)} + u_{i,j-1}^{(k)} + u_{i,j+1}^{(k)} \right)$$

Example Jacobi solver (cont.)

In pseudocode, it's roughly:

```
while not converged:
    for each grid point (i,j):
        u_new[i,j] = 0.25 * (u[i-1,j] + u[i+1,j] + u[i,j-1] + u[i,j+1])

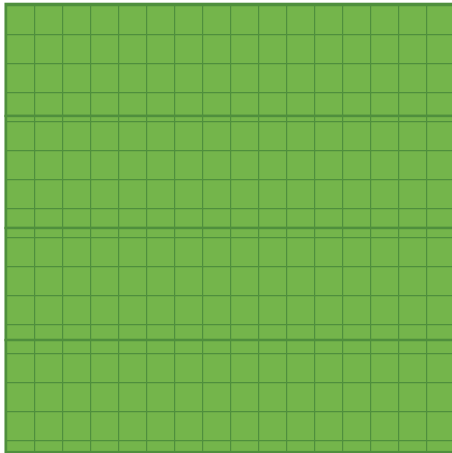
    # Apply periodic boundary conditions
    for each grid point i
        u_new[i,0] = u_new[i, ny-1]
        u_new[i,ny] = u_new[i, 1]

    swap(u, u_new)

    next iteration
```

Example: Domain decomposition

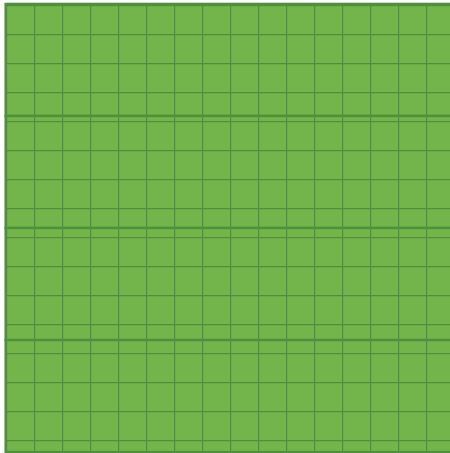
For simplicity, we will decompose the domain in the y direction only with MPI rank number increasing as you go down the domain (i.e. rank 0 at the top and rank $n-1$ at the bottom):



- In a realistic scenario, we would decompose the domain in both x and y directions to reduce the amount of communication as the number of MPI processes increases.

Example: Domain decomposition

For simplicity, we will decompose the domain in the y direction only with MPI rank number increasing as you go down the domain (i.e. rank 0 at the top and rank $n-1$ at the bottom):



- In a realistic scenario, we would decompose the domain in both x and y directions to reduce the amount of communication as the number of MPI processes increases.

Any questions?

Exercise 01: Jacobi solver

Use GPU-aware MPI to parallelize the Jacobi solvers across multiple GPUs. You will need to do the following:

1. Get the available GPU devices and use it and the local rank to set the active GPU for each process.
2. Compute the top and bottom neighbours (recall: periodic BCs on top/bottom).
3. Use `MPI_Sendrecv` with GPU-resident pointers/arrays to exchange data between the neighbors.

You can either choose CUDA+C++ or OpenMP+Fortran. Further details are provided in the respective README files.

There is an “advanced” extension task if you finish early.

Exercise 01: Solution

Setting the active GPU

```
234     int num_devices = 0;  
235     // TODO: Get the available GPU devices into `num_devices` and use it and the current rank to
```

Exercise 01: Solution

Setting the active GPU

```
234     int num_devices = 0;
235     // TODO: Get the available GPU devices into `num_devices` and use it and the current rank to
236     CUDA_RT_CALL(cudaGetDeviceCount(&num_devices));
237     CUDA_RT_CALL(cudaSetDevice(local_rank%num_devices));
```

Exercise 01: Solution

Computing the top and bottom neighbors

```
306      //TODO: Compute top and bottom neighbor, use reflecting/periodic boundaries. This means
307      // exchange data
308      const int top = 0;
309      const int bottom = 0;
```

Exercise 01: Solution

Computing the top and bottom neighbors

```
308      //TODO: Compute top and bottom neighbor, use reflecting/periodic boundaries. This means
309      // exchange data
310      const int top = rank > 0 ? rank - 1 : (size - 1);
311      const int bottom = (rank + 1) % size;
```

Exercise 01: Solution

Using `MPI_Sendrecv` to exchange data between neighbors

```
311      // TODO: Use MPI_Sendrecv to exchange the data with the top and bottom neighbors
312      // Use CUDA-aware MPI here, i.e. receive the data directly in a_new on the GPU and send
313      // without manually copying the data.
314
315      // The first newly calculated row ('iy_start') is sent to the top neighbor and the bottom
316      // ('iy_end') is received from the bottom process.
317      // The last calculated row ('iy_end-1') is sent to the bottom process and the top boundary
318      // Don't forget to synchronize the computation on the GPU before starting the data transfer
```

Exercise 01: Solution

Using `MPI_Sendrecv` to exchange data between neighbors

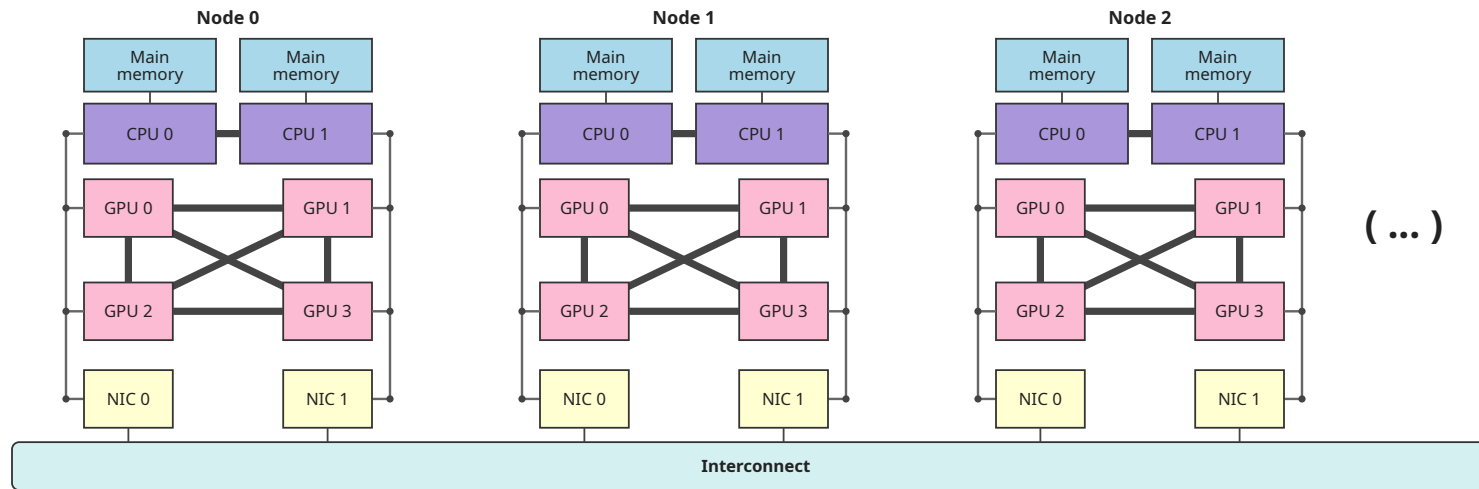
```
313      // TODO: Use MPI_Sendrecv to exchange the data with the top and bottom neighbors
314      // Use CUDA-aware MPI here, i.e. receive the data directly in a_new on the GPU and send
315      // without manually copying the data.
316
317      // The first newly calculated row ('iy_start') is sent to the top neighbor and the bottom
318      // ('iy_end') is received from the bottom process.
319      // The last calculated row ('iy_end-1') is sent to the bottom process and the top boundary
320      // Don't forget to synchronize the computation on the GPU before starting the data transfer
321      CUDA_RT_CALL(cudaEventSynchronize(compute_done));
322      PUSH_RANGE("MPI", 5)
323      MPI_CALL(MPI_Sendrecv(a_new + iy_start * nx, nx, MPI_REAL_TYPE, top, 0,
324                           a_new + (iy_end * nx), nx, MPI_REAL_TYPE, bottom, 0, MPI_COMM_WORLD,
325                           MPI_STATUS_IGNORE));
326      MPI_CALL(MPI_Sendrecv(a_new + (iy_end - 1) * nx, nx, MPI_REAL_TYPE, bottom, 0, a_new, r
327                           MPI_REAL_TYPE, top, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE));
328      POP_RANGE
```

Optimization techniques

Why optimization is *more* important on GPUs

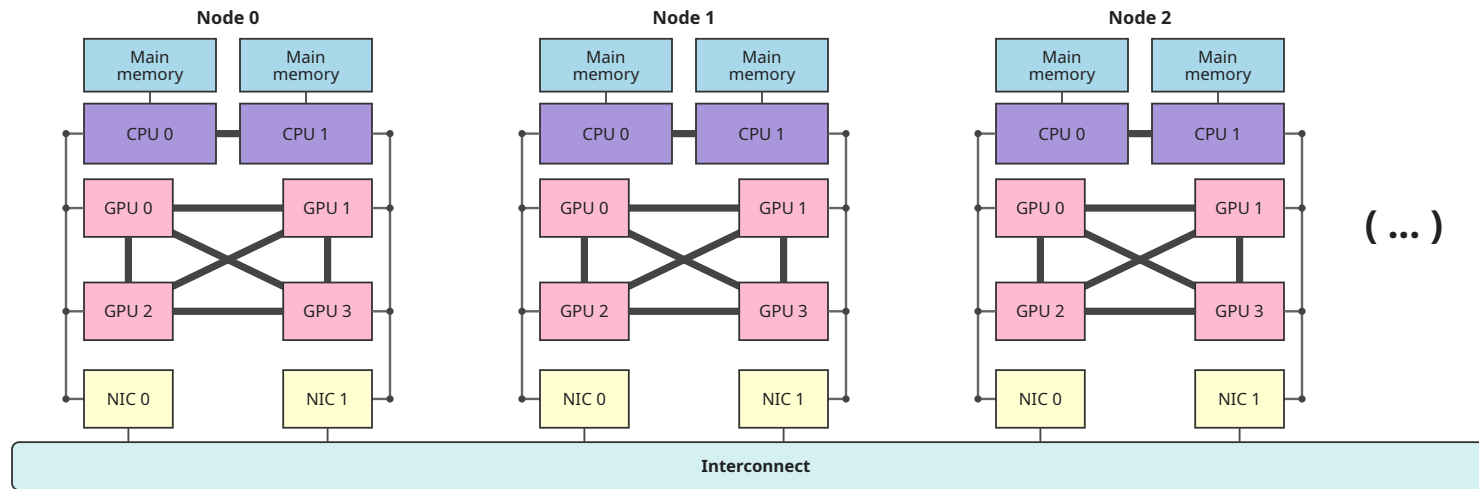
- Once you have offloaded heavy computations to the GPU, the remaining bottlenecks are often either:
 - Memory transfers between host and device.
 - Communication between GPUs.
- Keeping data GPU-resident as much as possible including by using GPU-aware MPI can help reduce memory transfer bottlenecks
- There's not usually a single “best” way to optimize communication between GPUs but there are some common techniques we will cover that may improve performance.
- Many are also relevant to CPU-only MPI applications but the relative performance gains may be more significant on GPUs.

NUMA, affinity and GPU distribution



- Note that in the node diagram, GPUs 0 and 2 are “closest” to CPU 0 and GPUs 1 and 3 are “closest” to CPU 1.
- Similarly, GPUs 0 and 1 are “closest” to NIC 0 and GPUs 2 and 3 are “closest” to NIC 1.
- This diagram is somewhat arbitrary and the actual topology differs between systems.
- Within just the CPU, for a given CPU cores, some memory domains are “closer” than others. This is known as Non-Uniform Memory Access (NUMA).
- The differences in distance can mean, for a specific GPU, it may be faster to transfer/read data to/from specific CPU cores and NICs compared to others.

NUMA, affinity and GPU distribution



- To achieve the best performance, it is often necessary to bind specific CPU cores, a specific GPU and a specific NIC to each MPI process.
- It is technically possible to figure out the topology yourself using tools such as `hwloc-ls` and `nvidia-smi topo` and then write a binding script but it is usually best and easiest to check the system documentation to see if one has been provided.
- Typically, these scripts are used in your MPI launch line:

```
mpirun <MPI options> ./binding_script.sh <application> <options>
```
- Huge potential speedups possible: I've observed a 3x speedup recently.

Example GPU binding script (CSD3 Ampere nodes)

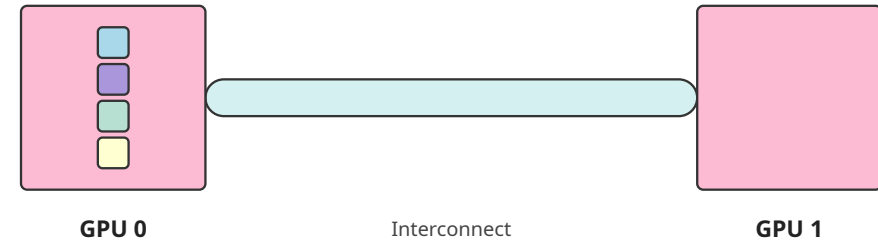
```
1 #!/bin/bash
2 GPUS=(0 1 2 3)
3
4 # associate GPUs 0,1 with MLX0, GPUs 2,3 with MLX1
5 NICS=(mlx5_0:1 mlx5_0:1 mlx5_1:1 mlx5_1:1)
6
7 # On the Ampere nodes we have 2x64 core CPUs, each organised into 4 NUMA domains
8 # We will use only a subset of the available NUMA domains, i.e. 1 NUMA domain per GPU
9 CPUS=(48-63 16-31 112-127 80-95)
10
11 # This is the list of memory domains we should use for each GPU
12 MEMS=(3 1 7 5)
13
14 lrank=$OMPI_COMM_WORLD_LOCAL_RANK
15
16 export CUDA_VISIBLE_DEVICES=${GPUS[${lrank}]}
17 export UCX_NET_DEVICES=${NICS[${lrank}]}
18 numactl --physcpubind="${CPUS[${lrank}]}" --membind="${MEMS[${lrank}]}" "$@"
```

Any questions?

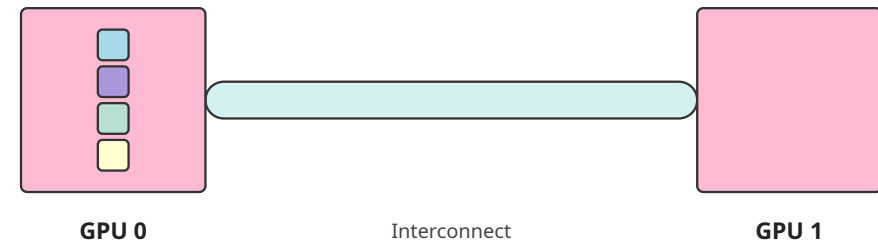
DiRAC Packing kernels

<https://tinyurl.com/dirac-multi-gpu>

- Since there is a greater overhead in launching GPU kernels, sending lots of small messages between GPUs is often more inefficient than doing the same in a CPU-only MPI application.
- It is usually more efficient to “pack” multiple small messages into a single larger buffer and send that instead.
- If you use MPI derived datatypes with non-contiguous data, the MPI implementation will usually not pack the data for you so communication will be inefficient → Replace with packing kernels.
- Since packing kernels will typically be quite small, it is best to launch them asynchronously (`nowait` clause for OpenMP and `taskwait` to synchronize).



Communication without packing

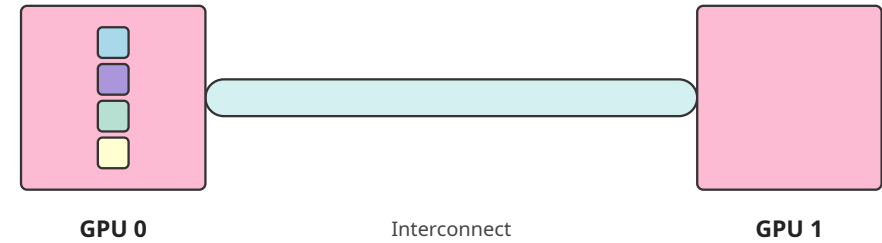


Communication with packing

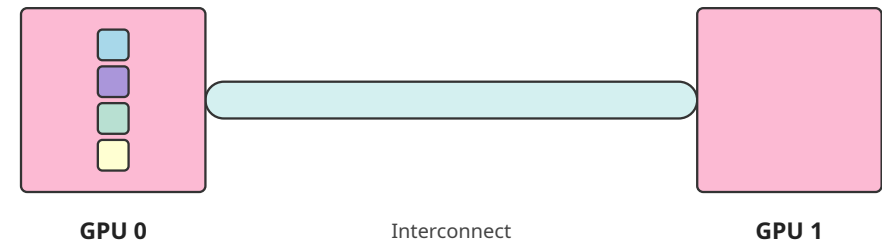
DiRAC Packing kernels

<https://tinyurl.com/dirac-multi-gpu>

- For each pair of communicating processes, you can achieve this as follows:
 1. Allocate a single contiguous GPU-resident buffer to send data and another to receive data.
 2. Launch a kernel to pack the data to be sent into the send buffer.
 3. Send the packed buffer to the other GPU using GPU-aware MPI.
 4. Launch a kernel to unpack the received buffer at the other end as required.



Communication without packing



Communication with packing

Other MPI optimization techniques

- You probably don't need `MPI_Barriers` for correctness. Unnecessary synchronization can be expensive.
- Where you can, use non-blocking communication routines.
 - These are usually prefixed with `MPI_I` e.g. `MPI_Isend`, `MPI_Irecv`.
 - These allow you to overlap communication with computation.
 - Remember to synchronize appropriately e.g. with `MPI_wait` before trying to access received data.
- Use MPI collective communication routines (e.g. `MPI_Allreduce`, `MPI_Bcast`, `MPI_Alltoall`) where possible (non-blocking versions available since MPI 3.0).
 - Many MPI implementations have optimized collective communication routines that can be significantly more efficient than implementing collectives naively with point-to-point calls.
 - Figuring out when you might be accidentally implementing collectives naively can be non-trivial.
 - Neighbourhood collectives may be more efficient for exchanging halo regions in a decomposed domain.

Any questions?

Exercise 02: Packing kernels and non-blocking communication

Replace multiple blocking messages with a packing kernel and non-blocking communication. The starting point performs a ring-shift communication between GPUs where each rank sends `num_messages` small non-contiguous messages to its right neighbour. You will need to do the following:

1. Implement the `pack_kernel` routine that gathers the non-contiguous messages into a contiguous buffer.
2. Implement the `unpack_kernel` routine that scatters a contiguous buffer back into the strided layout.
3. In the packing loop, call the necessary packing/unpacking kernels and the non-blocking MPI routines to send/receive the packed buffers. Remember to synchronize appropriately.

You can either choose CUDA+C++ or OpenMP+Fortran. Further details are provided in the respective README files.

There is an extension task if you finish early.

Exercise 02: Solution

Implementing the `pack_kernel`

```
93      ! TODO: Implement the pack kernel.
94      ! Each thread copies one element from the strided source buffer (stride, num_messages)
95      ! to the contiguous destination buffer (msg_size, num_messages):
96      !   dst(i, m) = src(i, m)   for i = 1..msg_size, m = 1..num_messages
97      ! Use a !$omp target teams distribute parallel do collapse(2) construct.
98      subroutine pack_kernel(dst, src, num_messages, msg_size, stride)
99          real(rk), intent(inout) :: dst(:, :)
100         real(rk), intent(in)  :: src(:, :)
101         integer, intent(in) :: num_messages, msg_size, stride
102         ! TODO
103     end subroutine pack_kernel
```

Exercise 02: Solution

Implementing the `pack_kernel`

```
93      ! Pack strided messages into a contiguous buffer.
94      subroutine pack_kernel(dst, src, num_messages, msg_size, stride)
95          real(rk), intent(inout) :: dst(:, :)
96          real(rk), intent(in) :: src(:, :)
97          integer, intent(in) :: num_messages, msg_size, stride
98          integer :: m, i
99          !$omp target teams distribute parallel do collapse(2)
100         do m = 1, num_messages
101             do i = 1, msg_size
102                 dst(i, m) = src(i, m)
103             end do
104         end do
105     end subroutine pack_kernel
```

Exercise 02: Solution

Implementing the `unpack_kernel`

```
105      ! TODO: Implement the unpack kernel.
106      ! Each thread copies one element from the contiguous source buffer (msg_size, num_messages)
107      ! to the strided destination buffer (stride, num_messages):
108      !   dst(i, m) = src(i, m)   for i = 1..msg_size, m = 1..num_messages
109      ! Use a !$omp target teams distribute parallel do collapse(2) construct.
110      subroutine unpack_kernel(dst, src, num_messages, msg_size, stride)
111          real(rk), intent(inout) :: dst(:, :)
112          real(rk), intent(in)  :: src(:, :)
113          integer, intent(in) :: num_messages, msg_size, stride
114          ! TODO
115      end subroutine unpack_kernel
```

Exercise 02: Solution

Implementing the `unpack_kernel`

```
107      ! Unpack a contiguous buffer back into strided messages.
108      subroutine unpack_kernel(dst, src, num_messages, msg_size, stride)
109          real(rk), intent(inout) :: dst(:, :)
110          real(rk), intent(in) :: src(:, :)
111          integer, intent(in) :: num_messages, msg_size, stride
112          integer :: m, i
113          !$omp target teams distribute parallel do collapse(2)
114          do m = 1, num_messages
115              do i = 1, msg_size
116                  dst(i, m) = src(i, m)
117              end do
118          end do
119      end subroutine unpack_kernel
```

Exercise 02: Solution

The packing loop

```
246     do iter = 1, num_iters
247         ! TODO: Call pack_kernel to gather send into send_buf
248         !   call pack_kernel(send_buf, send, num_messages, msg_size, stride)
249
250         ! TODO: Post MPI_Isend (send_buf -> next) and MPI_Irecv (recv_buf <- prev)
251         !   inside a !$omp target data use_device_addr(send_buf, recv_buf) region.
252         !   The signatures are
253         !       MPI_Isend(buf, count, datatype, dest, tag, comm, request, ierror)
254         !       MPI_Irecv(buf, count, datatype, source, tag, comm, request, ierror)
255         !   Then call MPI_Waitall to complete both requests:
256         !       MPI_Waitall(count, array_of_requests, array_of_statuses, ierror)
257
258         ! TODO: Call unpack_kernel to scatter recv_buf into recv
259         !   call unpack_kernel(recv, recv_buf, num_messages, msg_size, stride)
260     end do
```

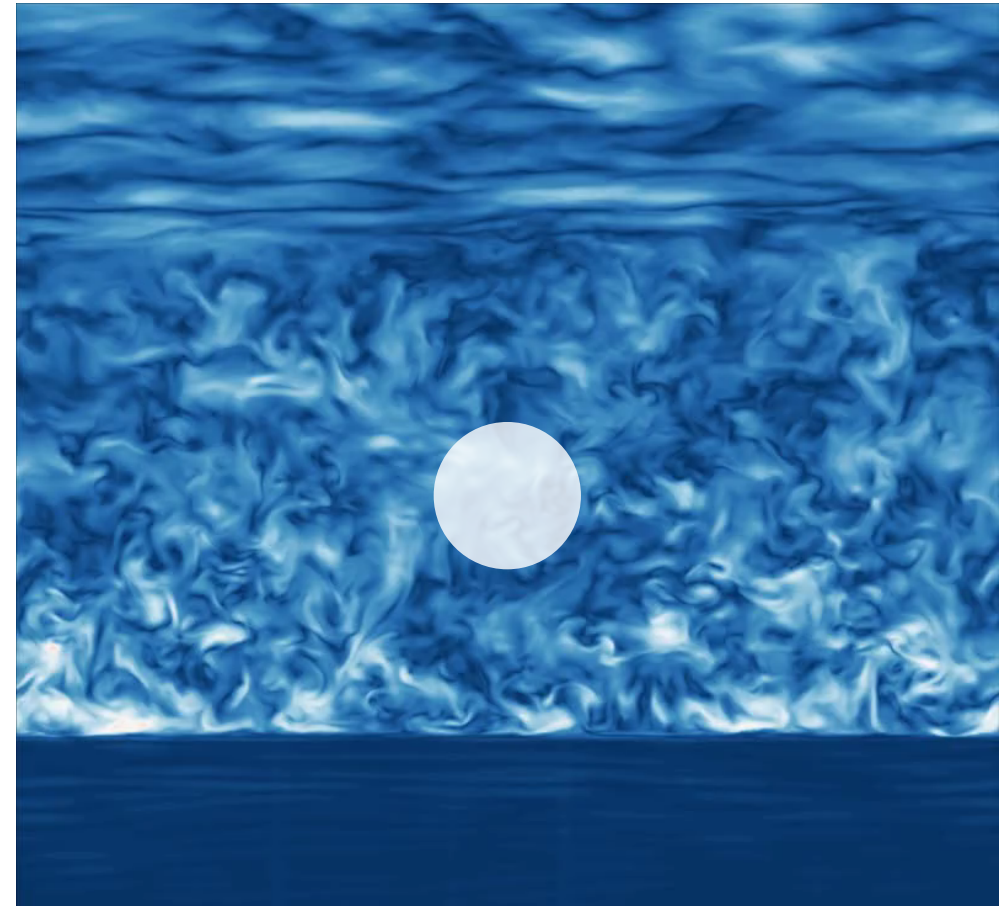
Exercise 02: Solution

The packing loop

```
250     do iter = 1, num_iters
251         call pack_kernel(send_buf, send, num_messages, msg_size, stride)
252
253         !$omp target data use_device_addr(send_buf, recv_buf)
254         call MPI_Isend(send_buf, packed_size, MPI_REAL_TYPE, next, 0, &
255                      MPI_COMM_WORLD, reqs(1), ierr)
256         call mpi_check(ierr, "MPI_Isend")
257         call MPI_Irecv(recv_buf, packed_size, MPI_REAL_TYPE, prev, 0, &
258                      MPI_COMM_WORLD, reqs(2), ierr)
259         call mpi_check(ierr, "MPI_Irecv")
260         call MPI_Waitall(2, reqs, statuses, ierr)
261         call mpi_check(ierr, "MPI_Waitall")
262         !$omp end target data
263
264         call unpack_kernel(recv, recv_buf, num_messages, msg_size, stride)
265     end do
```

Case Study: PROMPI

- PROMPI is a fluid dynamics code that is used to simulate complex hydrodynamic processes within stars.
- Code:
 - Fortran
 - MPI parellization
 - OpenACC for GPU acceleration (currently being ported to OpenMP)



0:00 / 0:21

Evolution of the velocity magnitude $|\mathbf{v}|$ for a 1024^3 simulation of the Carbon-burning shell

Starting place

How was communication handled previously?

Previously the code used:

- Nvidia managed memory extension to OpenACC:
 - When data access is attempted on the host (CPU)/device (GPU) but the data resides on the device/host, a page fault triggers the runtime to migrate the data across.
- MPI derived datatypes:
 - Data arrays have halo regions/ghost cells to allow calculating derivatives.
 - In each direction, these are non-contiguous in memory but regularly spaced.
 - **MPI_Type_vector** is designed to handle this type of memory layout:

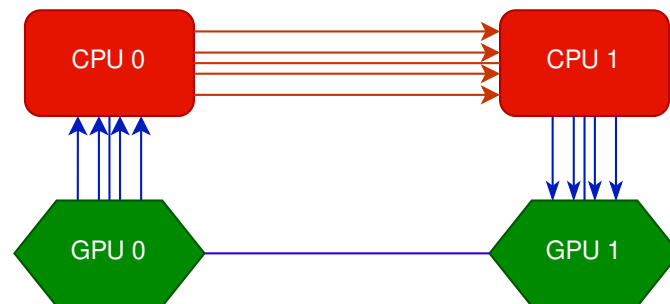


- Effectively blocking MPI calls:
 - The data for each variable is stored in separate arrays.
 - The ghost data for each array was sent in separate **MPI_Isends**.
 - However, **MPI_wait** was called after every **MPI_Irecv**.

The problem

How did this configuration lead to poor performance on multiple GPUs?

- Because of managed memory, each contiguous chunk of the `MPI_Type_vector` was separately migrated from device to host memory.
- These were then communicated using MPI from the host memory on 1 rank to another.
- The effectively blocking MPI calls meant these transfers were likely performed sequentially.
- Lots of small host-device transfers (visible in Nvidia Nsight Systems and Linaro Forge profiling) → poor performance.
- For a **512³** test simulation running on 8 Tursa Nvidia A100s (2 nodes), basic timer profiling showed that > 90% of the walltime was spent in communication.



The solution

How were the communications optimised?

I significantly refactored the communication in the following ways:

- Manual packing and unpacking of data:
 - Single send/receive buffer for each pair of communicating processes.
 - Asynchronous OpenACC kernels to [un]pack data from/to ghosts cells from all variables into the single buffer on the GPU.
 - No more **MPI_Type_vector**.
- Forced use of GPU-aware MPI:

```
1 !$acc host_data use_device(send_buf)
2 call mpi_isend(send_buf, ...)
```

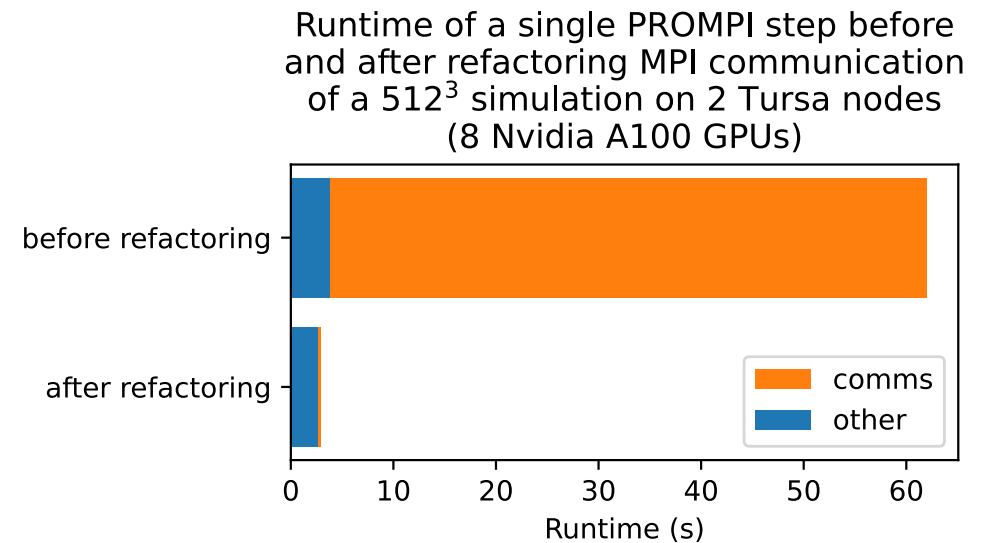
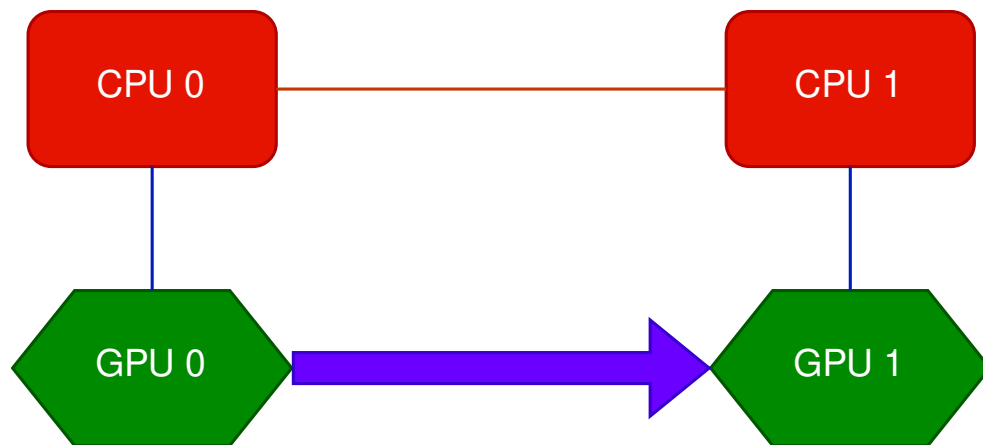
(analogous to **\$!omp target data use_device_addr(send_buf)** for OpenMP)

- **MPI_Waitall** after all sends and receives for each direction (truly asynchronous).

How much better is the performance following these changes?

After these changes with our test case:

- ~200x speed-up in communication leading to ~20x overall speed-up.
- < 10% of the walltime spent in communication.

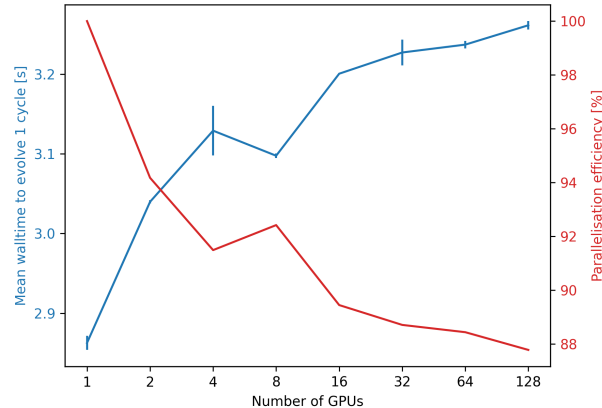


How does *PROMPI* scale after these improvements?

Weak scaling on Tursa

- Excellent weak scaling of 88% efficiency up to 128 GPUs.
- Most relevant scaling for group given typical research workflows.

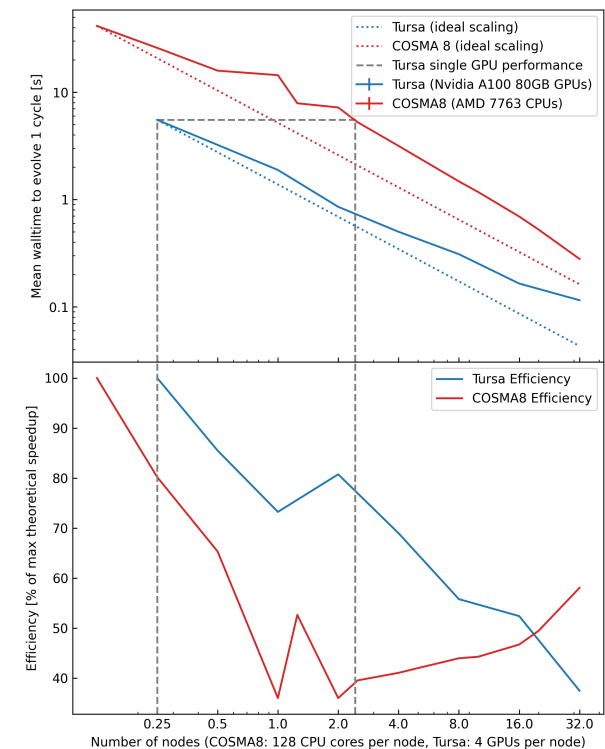
Weak scaling of the Kelvin Helmholtz TIMMES example with 256^3 per GPU



Strong scaling on Tursa and COSMA8

- Good strong scaling (>50% efficiency) up to around 32 Tursa Nvidia A100 80GB GPUs.
- Efficiency drops for greater numbers due to GPU underutilization.
- Grey line shows roughly how many COSMA8 (Milan) nodes are equivalent to 1 Tursa GPU.

Strong scaling with the Kelvin Helmholtz TIMMES example on a 320^3 domain

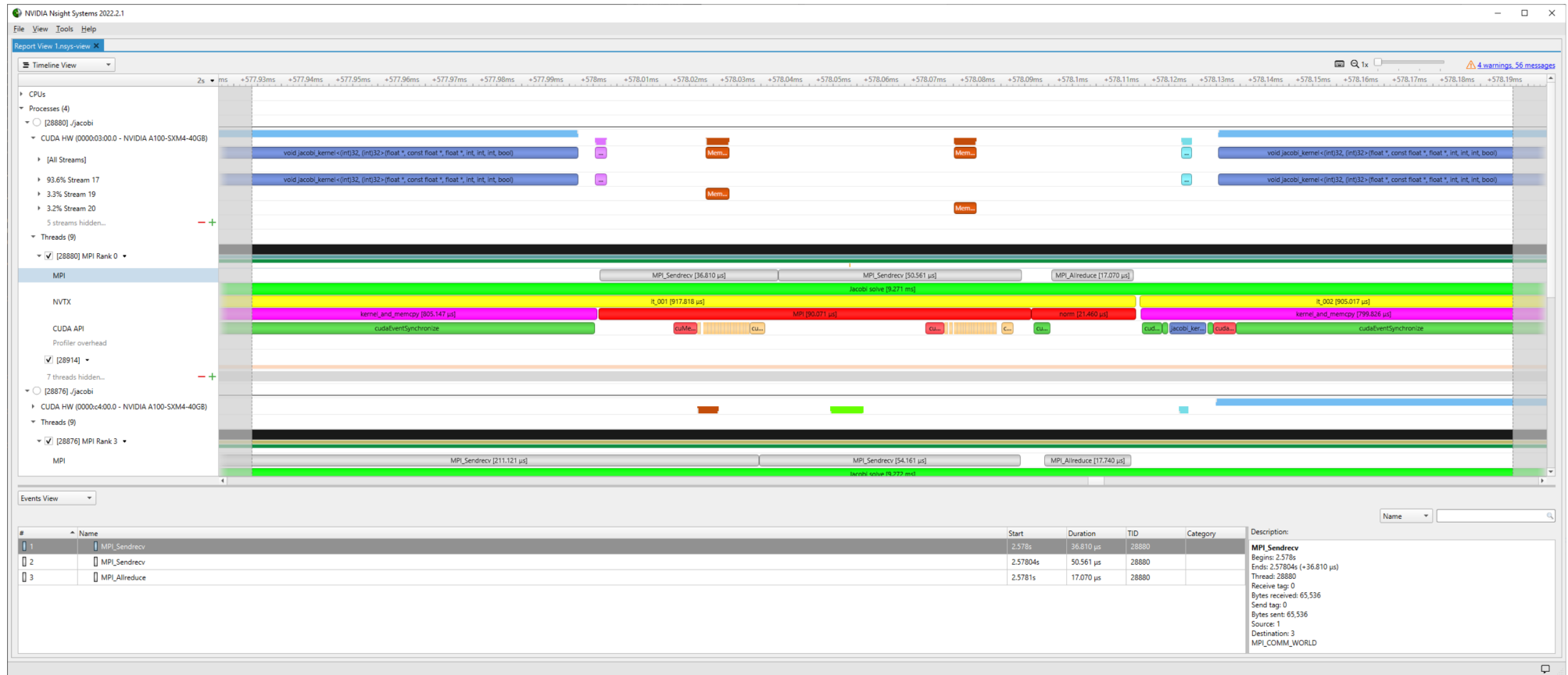


Any questions?

Overlapping computation and communication

Timeline profiling

Let's go back to the Jacobi solver. Profiling with Nsight Systems¹ gives the following timeline:



Overlapping computation with communication

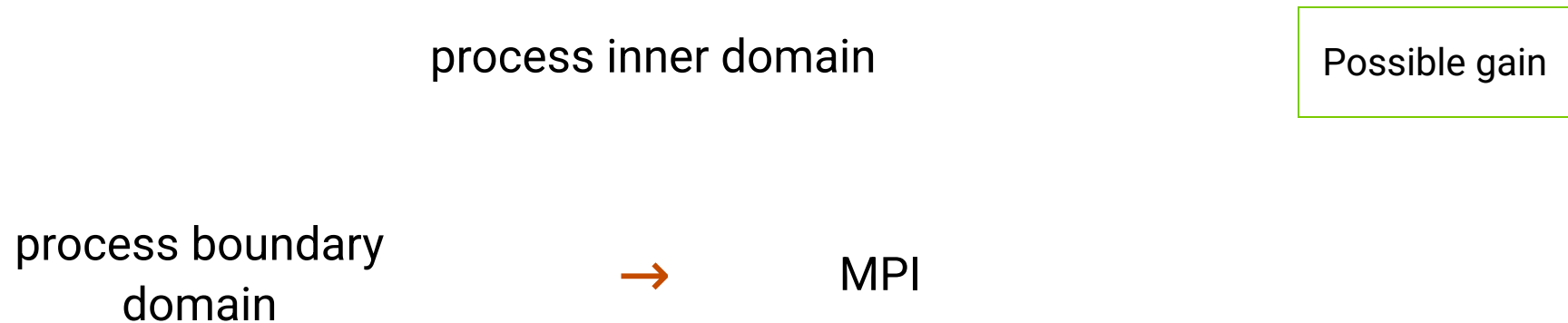
Currently the Jacobi solver computes the whole domain and then exchanges the boundary data with MPI:

process whole domain

MPI

Overlapping computation with communication

We can overlap computation with communication by computing the inner domain while exchanging the boundary data with MPI:



CUDA/HIP Streams (C++ only)

- A CUDA/HIP stream is an ordered queue of operations (e.g. kernel execution, memory copies, events, etc.) that execute on the GPU.
- Operations within a single stream are guaranteed to execute in order but operations in different streams may execute concurrently.
- By default if you don't specify a stream, CUDA/HIP operations uses the default (null) stream which is synchronous: it blocks operations on all other streams until it finishes which prevents concurrency.
- Streams can be created and synchronized as follows:

```
1 cudaStream_t my_new_stream;  
2 cudaStreamCreate(&my_new_stream);  
3 ...  
4 cudaStreamSynchronize(my_new_stream);
```

Separating inner and boundary domain computations in the Jacobi solver

In our Jacobi solver example, we can create new streams and separate the inner and boundary domain computations:

```
1 launch_jacobi_kernel(a_new, a, l2_norm_d, iy_start, iy_end, nx, calculate_norm,  
2                       compute_stream);  
3 const int top = rank > 0 ? rank - 1 : (size - 1);  
4 const int bottom = (rank + 1) % size;  
5  
6 CUDA_RT_CALL(cudaStreamSynchronize(compute_stream));  
7 MPI_CALL(MPI_Sendrecv(a_new + iy_start * nx, nx, MPI_REAL_TYPE, top, 0,  
8                      a_new + (iy_end * nx), nx, MPI_REAL_TYPE, bottom, 0, MPI_COMM_WORLD,  
9                      MPI_STATUS_IGNORE));  
10 MPI_CALL(MPI_Sendrecv(a_new + (iy_end - 1) * nx, nx, MPI_REAL_TYPE, bottom, 0, a_new, nx,  
11                      MPI_REAL_TYPE, top, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE));
```

Separating inner and boundary domain computations in the Jacobi solver

In our Jacobi solver example, we can create new streams and separate the inner and boundary domain computations:

```
1 launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_start + 1), (iy_end - 1), nx,
2                       calculate_norm, compute_stream);
3 launch_jacobi_kernel(a_new, a, l2_norm_d, iy_start, (iy_start + 1), nx, calculate_norm,
4                       push_top_stream);
5 launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_end - 1), iy_end, nx, calculate_norm,
6                       push_bottom_stream);
7 const int top = rank > 0 ? rank - 1 : (size - 1);
8 const int bottom = (rank + 1) % size;
9
10 CUDA_RT_CALL(cudaStreamSynchronize(push_top_stream));
11 MPI_CALL(MPI_Sendrecv(a_new + iy_start * nx, nx, MPI_REAL_TYPE, top, 0,
12                       a_new + (iy_end * nx), nx, MPI_REAL_TYPE, bottom, 0, MPI_COMM_WORLD,
13                       MPI_STATUS_IGNORE));
14
15 CUDA_RT_CALL(cudaStreamSynchronize(push_bottom_stream));
16 MPI_CALL(MPI_Sendrecv(a_new + (iy_end - 1) * nx, nx, MPI_REAL_TYPE, bottom, 0, a_new, nx,
17                       MPI_REAL_TYPE, top, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE));
```

Note the order here is intentional. The inner kernel is largest and will take the longest to complete so we want to start it first.

Streams with Priority

It's also possible to create streams with different priorities. CUDA has the following functions:

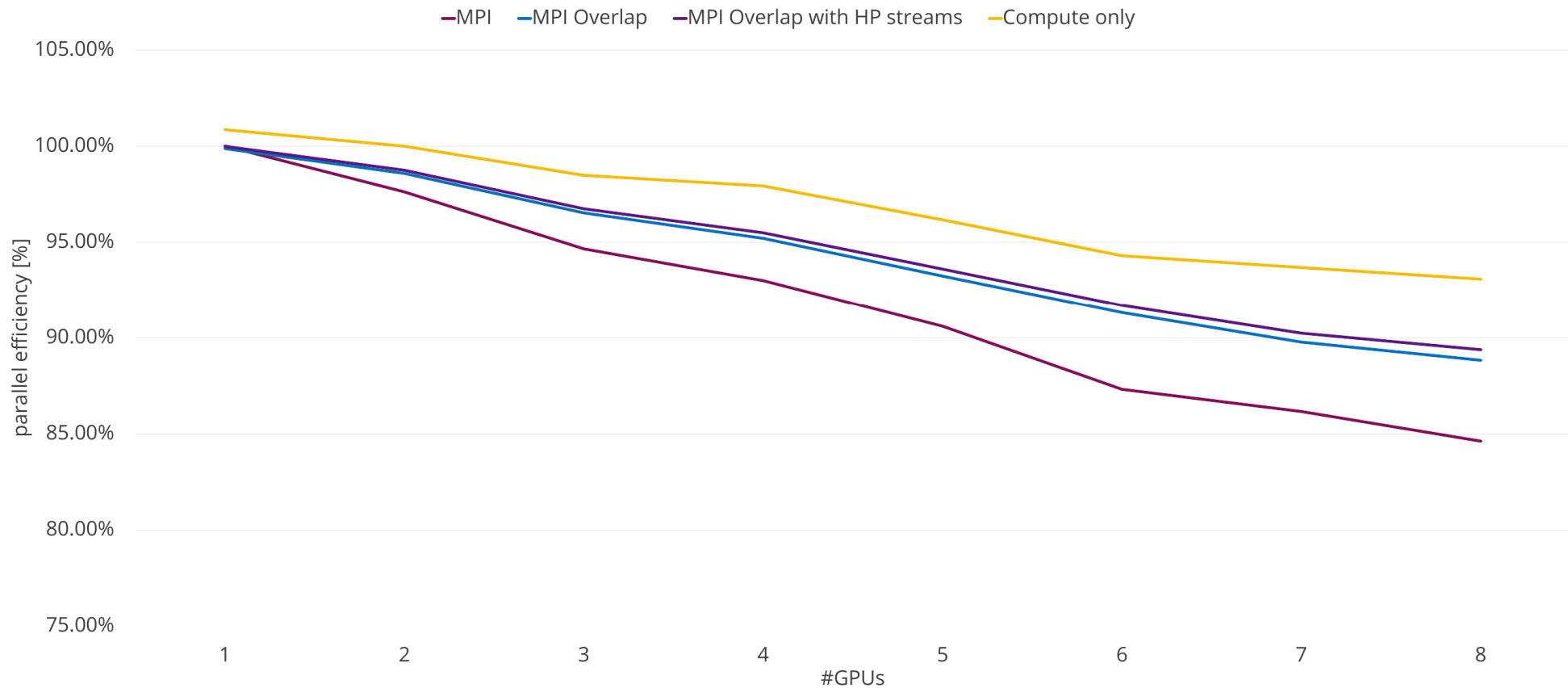
```
cudaError_t cudaStreamCreateWithPriority(cudaStream_t *stream, unsigned int flags, int priority);  
  
cudaError_t cudaDeviceGetStreamPriorityRange(int *leastPriority, int *greatestPriority);
```

In our Jacobi example, we can make the boundary streams higher priority than the inner stream so that they finish earlier and communication can start sooner.

Any questions?

Scaling improvement

After making these changes, improved scaling is observed in the Jacobi solver example¹



¹ This screenshot is taken from the ISC/SC Multi-GPU tutorial slides and was generated on the J1WFLS Booster system with Nvidia A100 40GB GPUs

Exercise 03: Overlapping communication and computation

Overlap the MPI halo exchange with the Jacobi computation using multiple CUDA streams. The starting point is a multi-GPU Jacobi solver where the halo exchange is already implemented with GPU-aware non-blocking MPI, but the whole Jacobi kernel must finish before any communication starts. You will need to do the following:

1. Query the CUDA stream priority range and create additional top and bottom streams with the greatest priority (and the compute stream with the least).
2. Split the Jacobi kernel launch into a bulk computation on the compute stream and two one-row boundary kernels on the top and bottom streams.
3. Synchronize the compute stream on the boundary events before copying the norm, and synchronize on the boundary streams (not the compute stream) before posting the MPI exchanges.
4. Destroy the additional streams and events before the program ends.

This is a CUDA+C++ only exercise. Since we are only using a small number of GPUs, there won't be much speed-up and we won't be able to test scaling. However you can observe the benefits by profiling with Nsight Systems. Further details are in the README file.

Exercise 03: Solution

Querying stream priorities and creating streams

```
274 // TODO: Query the CUDA stream priority range (least/greatest) with
275 // cudaDeviceGetStreamPriorityRange and declare the additional top and bottom
276 // CUDA streams (push_top_stream, push_bottom_stream) and their corresponding
277 // events (push_top_done, push_bottom_done)
278 cudaStream_t compute_stream;
279
280 // TODO: Create the additional top and bottom CUDA streams with the greatest
281 // available priority and change the creation of compute_stream to use the
282 // least available priority, using cudaStreamCreateWithPriority
283 cudaStreamCreate(&compute_stream)
```

(CUDA_RT_CALL()s omitted for brevity.)

Exercise 03: Solution

Querying stream priorities and creating streams

```
274     int leastPriority = 0;
275     int greatestPriority = leastPriority;
276     cudaDeviceGetStreamPriorityRange(&leastPriority, &greatestPriority)
277
278     cudaStream_t compute_stream;
279     cudaStream_t push_top_stream;
280     cudaStream_t push_bottom_stream;
281
282     cudaStreamCreateWithPriority(&compute_stream, cudaStreamDefault, leastPriority)
283     cudaStreamCreateWithPriority(&push_top_stream, cudaStreamDefault, greatestPriority)
284     cudaStreamCreateWithPriority(&push_bottom_stream, cudaStreamDefault, greatestPriority)
285
286     cudaEvent_t push_top_done;
287     cudaEventCreateWithFlags(&push_top_done, cudaEventDisableTiming)
288     cudaEvent_t push_bottom_done;
289     cudaEventCreateWithFlags(&push_bottom_done, cudaEventDisableTiming)
```

(CUDA_RT_CALL()s omitted for brevity.)

Exercise 03: Solution

Splitting the Jacobi kernel launch

```
344      // TODO: Modify the launch of the Jacobi kernel so that the bulk of the local
345      // domain (rows iy_start+1 to iy_end-1) is computed on the compute stream and
346      // launch two additional one-row kernels computing the top (row iy_start) and
347      // bottom (row iy_end-1) boundary rows on the top and bottom streams. Make
348      // these streams wait on the reset_l2norm_done event before launching with
349      // cudaStreamWaitEvent (as all three kernels atomically accumulate into
350      // l2_norm_d which needs to be zeroed first) and record the push_top_done and
351      // push_bottom_done events on their streams after each launch.
352      launch_jacobi_kernel(a_new, a, l2_norm_d, iy_start, iy_end, nx, calculate_norm,
353                          compute_stream);
354      cudaEventRecord(compute_done, compute_stream)
```

(`CUDA_RT_CALL()`s omitted for brevity.)

Exercise 03: Solution

Splitting the Jacobi kernel launch

```
344     launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_start + 1), (iy_end - 1), nx,
345                           calculate_norm, compute_stream);
346     cudaEventRecord(compute_done, compute_stream)
347
348     cudaStreamWaitEvent(push_top_stream, reset_l2norm_done, 0)
349     launch_jacobi_kernel(a_new, a, l2_norm_d, iy_start, (iy_start + 1), nx, calculate_norm,
350                           push_top_stream);
351     cudaEventRecord(push_top_done, push_top_stream)
352
353     cudaStreamWaitEvent(push_bottom_stream, reset_l2norm_done, 0)
354     launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_end - 1), iy_end, nx, calculate_norm,
355                           push_bottom_stream);
356     cudaEventRecord(push_bottom_done, push_bottom_stream)
```

(CUDA_RT_CALL()s omitted for brevity.)

Exercise 03: Solution

Waiting on boundary events before the norm copy

```
357          // TODO: Make the compute stream wait on the push_top_done and
358          // push_bottom_done events with cudaStreamWaitEvent before copying the norm
359          // to the host, as the boundary kernels also accumulate into l2_norm_d
360          cudaMemcpyAsync(l2_norm_h, l2_norm_d, sizeof(real), cudaMemcpyDeviceToHost,
361                          compute_stream);
```

(**CUDA_RT_CALL**())s omitted for brevity.)

Exercise 03: Solution

Waiting on boundary events before the norm copy

```
357         cudaStreamWaitEvent(compute_stream, push_top_done, 0);  
358         cudaStreamWaitEvent(compute_stream, push_bottom_done, 0);  
359         cudaMemcpyAsync(l2_norm_h, l2_norm_d, sizeof(real), cudaMemcpyDeviceToHost,  
360                        compute_stream);
```

(CUDA_RT_CALL()s omitted for brevity.)

Exercise 03: Solution

Synchronizing on boundary streams before MPI exchanges

```
366      // TODO: Modify the synchronisation on the compute stream to be a
367      // synchronisation on the top stream, so that only the top boundary kernel
368      // needs to be done before the top exchange is posted
369      cudaEventSynchronize(compute_done);
370      MPI_Irecv(a_new + iy_end * nx, nx, MPI_REAL_TYPE, bottom, 0, MPI_COMM_WORLD,
371              &requests[0]);
372      MPI_Isend(a_new + iy_start * nx, nx, MPI_REAL_TYPE, top, 0, MPI_COMM_WORLD,
373              &requests[1]);
374
375      // TODO: Add a synchronisation on the bottom stream before the bottom exchange,
376      // so that only the bottom boundary kernel needs to be done before it is posted
377      MPI_Irecv(a_new, nx, MPI_REAL_TYPE, top, 1, MPI_COMM_WORLD, &requests[2]);
378      MPI_Isend(a_new + (iy_end - 1) * nx, nx, MPI_REAL_TYPE, bottom, 1, MPI_COMM_WORLD,
379              &requests[3]);
```

(`CUDA_RT_CALL()`s and `MPI_CALL()`s omitted for brevity.)

Exercise 03: Solution

Synchronizing on boundary streams before MPI exchanges

```
366         cudaStreamSynchronize(push_top_stream);
367         MPI_Irecv(a_new + iy_end * nx, nx, MPI_REAL_TYPE, bottom, 0, MPI_COMM_WORLD,
368                 &requests[0]);
369         MPI_Isend(a_new + iy_start * nx, nx, MPI_REAL_TYPE, top, 0, MPI_COMM_WORLD,
370                 &requests[1]);
371
372         cudaStreamSynchronize(push_bottom_stream);
373         MPI_Irecv(a_new, nx, MPI_REAL_TYPE, top, 1, MPI_COMM_WORLD, &requests[2]);
374         MPI_Isend(a_new + (iy_end - 1) * nx, nx, MPI_REAL_TYPE, bottom, 1, MPI_COMM_WORLD,
375                 &requests[3]);
```

(`CUDA_RT_CALL()`s and `MPI_CALL()`s omitted for brevity.)

Exercise 03: Solution

Destroying additional streams and events

```
440    // TODO: Destroy the additional top and bottom CUDA streams and their
441    // corresponding events
442    cudaEventDestroy(compute_done);
443    cudaEventDestroy(reset_l2norm_done);
444    cudaStreamDestroy(compute_stream);
```

(`CUDA_RT_CALL( )`s omitted for brevity.)

Exercise 03: Solution

Destroying additional streams and events

```
440     cudaEventDestroy(push_bottom_done);  
441     cudaEventDestroy(push_top_done);  
442     cudaStreamDestroy(push_bottom_stream);  
443     cudaStreamDestroy(push_top_stream);  
444     cudaEventDestroy(compute_done);  
445     cudaEventDestroy(reset_l2norm_done);  
446     cudaStreamDestroy(compute_stream);
```

(`CUDA_RT_CALL( )`s omitted for brevity.)

Conclusion

What we've covered

- **GPU-aware MPI:** passing GPU pointers directly to MPI routines, with efficient data transfer via P2P and RDMA.
- **Optimization techniques:**
 - NUMA, affinity, and GPU/NIC binding for best hardware utilization.
 - Packing kernels to combine small non-contiguous messages into a single larger buffer.
 - Non-blocking communication and collectives.
- **Overlapping computation and communication:** using CUDA streams and events to improve scaling by exploiting concurrency between computation and communication.

What we haven't covered

- Advanced MPI features (e.g. persistent communication, partitioned communication, etc.)
- Collective Communication Libraries (Nvidia: NCCL, AMD: RCCL, Intel: oneCCL)
 - These are commonly used in deep learning frameworks.
 - We have used explicit synchronization between GPU kernels and MPI communication.
 - NCCL/RCCL/oneCCL are stream/queue-aware so can overlap communication and computation without explicit synchronization → more efficient.
 - They also provide optimized collective communication routines for GPUs.
 - They can be used alongside MPI.
- *SHMEM (Nvidia: NVSHMEM, AMD: rocSHMEM, Intel: Intel SHMEM)
 - These are PGAS (Partitioned Global Address Space) libraries that provide direct access to remote GPU memory.
 - They can be used alongside MPI.
 - They offer GPU-initiated communication and synchronization, which means communication can occur within a GPU kernel (saves on launch overhead).

Additional resources and references

- Material for this course has been adapted from the **ISC/SC tutorial on “Efficient Distributed GPU Programming for Exascale”**
 - This course also covers NCCL and NVSHMEM.
- Nvidia’s **multi-gpu-programming-models** repository contains various implementations of the Jacobi solver implemented with a variety of different communication models (MPI, NCCL, NVSHMEM, etc.) and optimization techniques.
- **ISC 2025 Advanced MPI course**

Any questions?